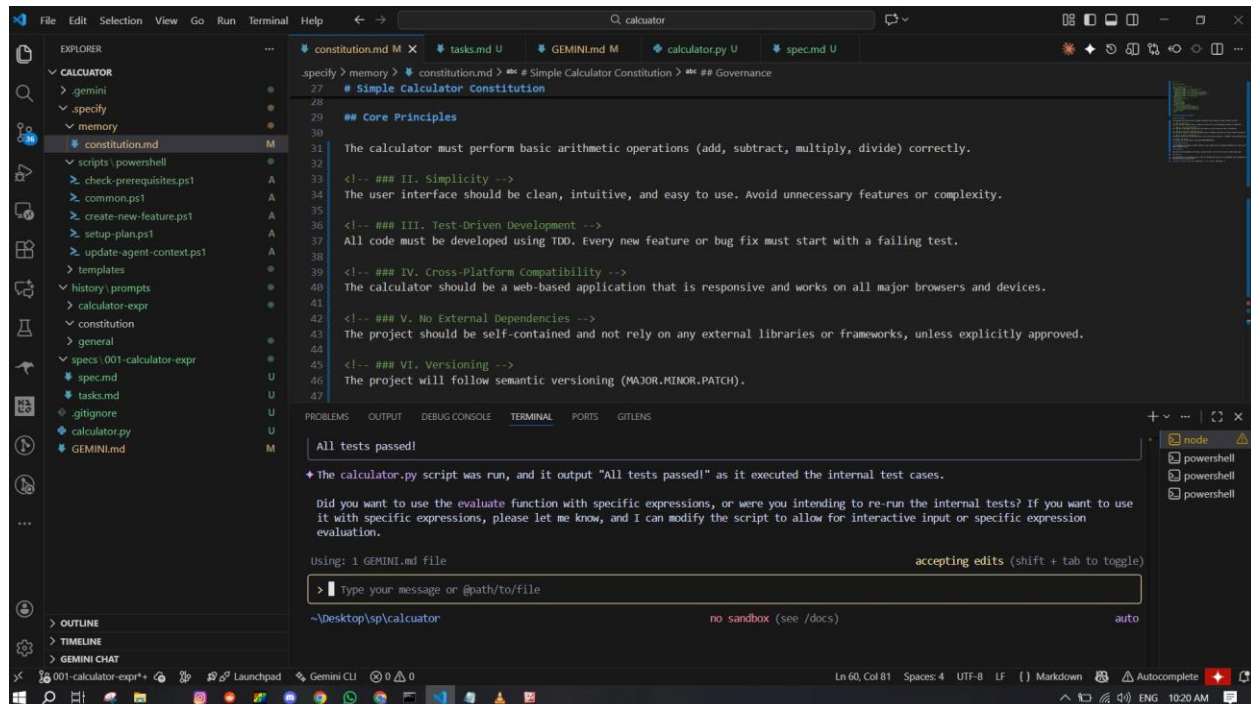


Prepared by: Muhammad Sarim Roll No: 00186281 Class Timing: Friday 6–9 PM

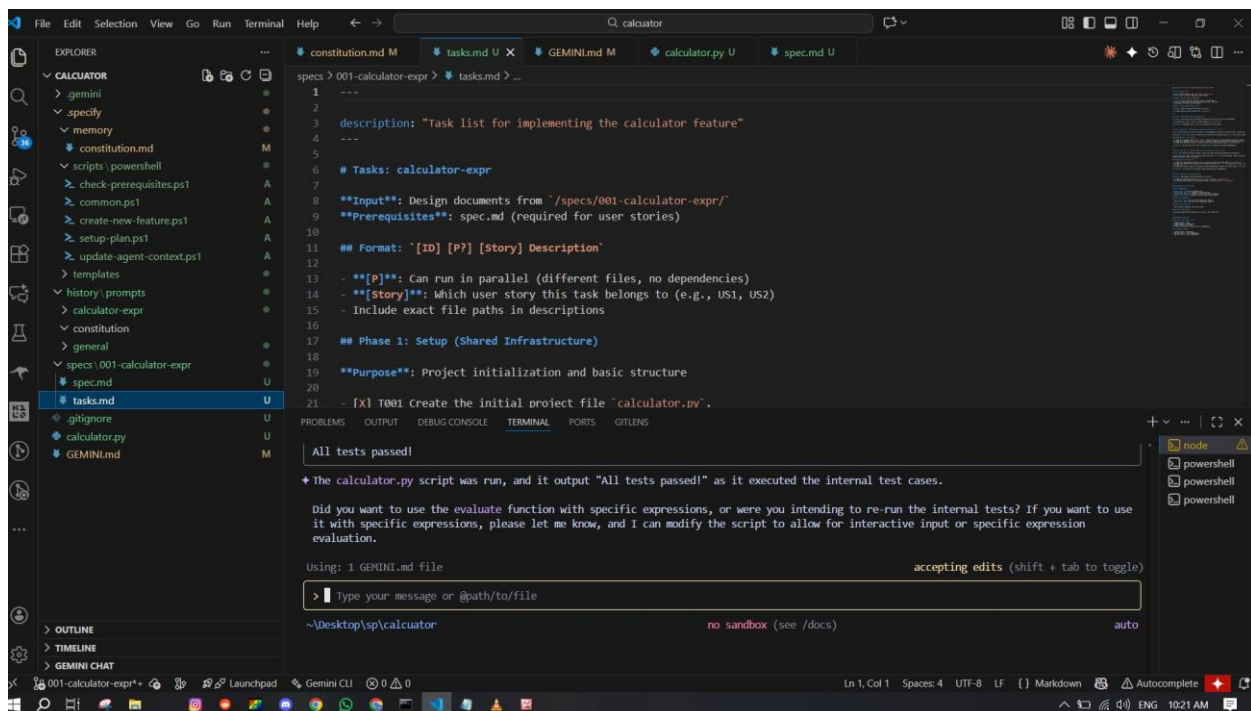


The screenshot shows the VS Code editor with the `constitution.md` file open. The file contains the following content:

```
specify > memory > constitution.md > ## Simple Calculator Constitution > ## Governance
# Simple Calculator Constitution
## Core Principles
The calculator must perform basic arithmetic operations (add, subtract, multiply, divide) correctly.
<!-- ## II. Simplicity -->
The user interface should be clean, intuitive, and easy to use. Avoid unnecessary features or complexity.
<!-- ## III. Test-Driven Development -->
All code must be developed using TDD. Every new feature or bug fix must start with a failing test.
<!-- ## IV. Cross-Platform Compatibility -->
The calculator should be a web-based application that is responsive and works on all major browsers and devices.
<!-- ## V. No External Dependencies -->
The project should be self-contained and not rely on any external libraries or frameworks, unless explicitly approved.
<!-- ## VI. Versioning -->
The project will follow semantic versioning (MAJOR.MINOR.PATCH).
```

The terminal output shows:

```
All tests passed!
+ The calculator.py script was run, and it output "All tests passed!" as it executed the internal test cases.
Did you want to use the evaluate function with specific expressions, or were you intending to re-run the internal tests? If you want to use it with specific expressions, please let me know, and I can modify the script to allow for interactive input or specific expression evaluation.
Using: 1 GEMINI.md file
accepting edits (shift + tab to toggle)
> Type your message or @path/to/file
~\Desktop\sp\calculator no sandbox (see /docs) auto
```



The screenshot shows the VS Code editor with the `tasks.md` file open. The file contains the following content:

```
specs > 001-calculator-expr > tasks.md > ...
1 ---
2
3 description: "Task list for implementing the calculator feature"
4 ---
5
6 ## Tasks: calculator-expr
7
8 **Input**: Design documents from `./specs/001-calculator-expr/`
9 **Prerequisites**: spec.md (required for user stories)
10
11 ## Format: `[ID] [P?] [Story] Description`
12
13 - **[P]**: Can run in parallel (different files, no dependencies)
14 - **[Story]**: Which user story this task belongs to (e.g., US1, US2)
15 - Include exact file paths in descriptions
16
17 ## Phase 1: Setup (Shared Infrastructure)
18
19 **Purpose**: Project initialization and basic structure
20
21 - [X] T001 Create the initial project file `calculator.py`.
```

The terminal output shows:

```
All tests passed!
+ The calculator.py script was run, and it output "All tests passed!" as it executed the internal test cases.
Did you want to use the evaluate function with specific expressions, or were you intending to re-run the internal tests? If you want to use it with specific expressions, please let me know, and I can modify the script to allow for interactive input or specific expression evaluation.
Using: 1 GEMINI.md file
accepting edits (shift + tab to toggle)
> Type your message or @path/to/file
~\Desktop\sp\calculator no sandbox (see /docs) auto
```

